

DIGITAL FORENSICS & INCIDENT RESPONSE

Silent Breach

Investigation Report

Prepared By: Dnyaneshwari Kale

Case Reference: CyberDefenders Blue Team CTF — “Silent Breach”

Subject Host: ethanPC | Subject User: ethan

Evidence Sources: AD1 logical image, browser history, Windows Mail (Hx) store, malware binary

Prepared using: FTK Imager, DB Browser for SQLite, PowerShell, CyberChef

Classification: For Training / Educational Use

1. Executive Summary

On the subject host “ethanPC”, the user “ethan” downloaded and executed a file masquerading as a PDF (IMF-Info.pdf.exe) that was in fact a Node.js application packaged into a Windows executable using the pkg bundler. Once run, the program silently wrote an obfuscated PowerShell script to disk and executed it, which used AES-256-CBC encryption (key derived via PBKDF2/Rfc2898DeriveBytes) to encrypt two sensitive PDF files on the user's Desktop and then deleted the originals — behavior consistent with a ransomware-style file-encryption payload.

The investigation reconstructed the full attack chain from disk image and application artifacts: the download event (browser history and Zone.Identifier ADS), the delivery application (Microsoft Edge), related indicators of a wider compromise (three at-risk server IPs referenced in a Windows Mail message), the malware's internal logic (recovered via static string analysis of the packaged Node.js binary), and finally the encryption password and routine — which was used to reverse the encryption and recover the secret flag hidden inside one of the encrypted documents.

The table below summarizes each investigative question and the final answer reached:

#	Question	Answer
1	MD5 hash of the malicious EXE	336A7CF476EBC7548C93507339196ABB
2	Download URL	http://192.168.16.128:8000/IMF-Info.pdf.exe
3	Download application	Microsoft Edge
4	At-risk server IPs (Windows Mail)	145.67.29.88, 192.168.16.128, 212.33.10.112
5	Predefined encryption password	Imf!nfo#2025Sec\$
6	Recovered secret (flag)	CyberDefenders{N3v3r_eX3cuTe_F!l3\$_d0wnL0ded_fr0m_M@lic10u5_\$erv3r}

2. Scope, Evidence & Methodology

2.1 Evidence Provided

- ethanPC - Copy.e01 — EnCase forensic image of the subject host, and ethanPC.ad1 — an FTK logical evidence file of the same host (located under Silent_Breach\temp_extract_dir\).
- History and History.copy0 — browser SQLite history databases (Chrome and Edge profiles respectively) recovered from the image.
- HxStore.hxd — the Windows Mail app's Hx (Universal Store) ESE database, containing cached mail artifacts.
- IMF-Info.pdf.exe — the suspected malicious executable recovered from the user's Downloads folder.
- IMF-Secret.enc and IMF-Mission.enc — AES-encrypted remnants of two PDF documents found on the subject's Desktop within the disk image.

2.2 Tools Used and Why

Tool	Purpose / Why Used
FTK Imager	To mount and browse the .E01/.AD1 forensic images without altering evidence, inspect file metadata (MZ header, timestamps, physical location on disk), and export files of interest (encrypted PDFs, Zone.Identifier ADS) for offline analysis.
DB Browser for SQLite	To open and query the Chrome/Edge History SQLite databases directly with SQL, since browser download and URL records are stored relationally across the downloads and downloads_url_chains tables.
Windows PowerShell	As the primary analysis shell: computing the MD5 hash of the executable (Get-FileHash), searching binary content for embedded strings and IOCs (Select-String with regex), and — once the malware's logic was understood — re-implementing the AES-256-CBC/PBKDF2 decryption natively using .NET cryptography classes to recover the original PDFs.
CyberChef	To reverse the malware's string-obfuscation (character reversal) and Base64-decode the resulting blob, revealing the embedded PowerShell encryption script in plaintext without needing to execute any part of the malware.
Static string analysis (Select-String against the EXE)	Because the executable is a Node.js application bundled with pkg (confirmed via embedded build paths referencing pkg.<hash>\node\src\...), the application logic — including the call to powershell.exe -ExecutionPolicy Bypass -File and the obfuscated payload string — exists as readable text inside the binary and did not require dynamic execution or sandbox detonation to recover.

2.3 Approach

Consistent with the Notes for Students guidance in the challenge brief, the artifacts were worked through in order, from disk image, to browser/mail history, to the executable itself, to the embedded PowerShell — and every IOC and technique was recorded as it was found, for the final IOC list and MITRE ATT&CK mapping in Sections 4 and 5.

Throughout the analysis, the malicious executable was never run. All findings on its behavior were obtained through static string extraction and manual deobfuscation of the embedded script, which is a safer and fully sufficient approach for this class of dropper.

3. Investigative Findings

3.1 Question 1 — Malicious File Hash

The suspicious executable, IMF-Info.pdf.exe, was recovered from the user's Downloads folder within the mounted evidence and hashed locally to obtain a unique fingerprint suitable for threat-intel lookups and IOC reporting.

```
Get-FileHash ".\IMF-Info.pdf.exe" -Algorithm MD5
```

```

PS C:\Users\hp\OneDrive\Documents\Desktop\Capstone_Challenges\TEST 2> Get-FileHash ".\IMF-Info.pdf.exe" -Algorithm MD5

Algorithm      Hash                                          Path
-----
MD5            336A7CF476EBC7548C93507339196ABB          C:\Users\hp\OneDrive\Documents\Desktop\Cap...

PS C:\Users\hp\OneDrive\Documents\Desktop\Capstone_Challenges\TEST 2> |

```

Figure 1 — MD5 hash computed for IMF-Info.pdf.exe

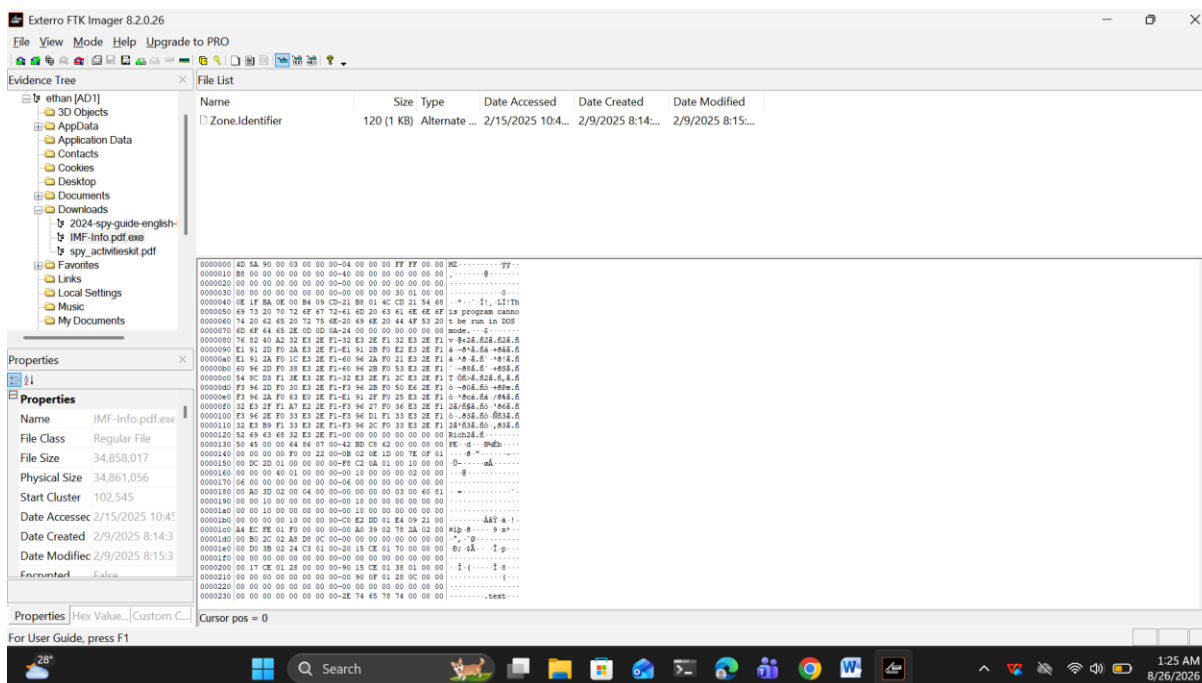


Figure 2 — FTK Imager view of IMF-Info.pdf.exe: file properties and MZ (PE) header confirming it is a genuine Windows executable, not a PDF, despite its filename.

Question 1

What is the MD5 hash of the potentially malicious EXE file the user downloaded?

Answer: **336A7CF476EBC7548C93507339196ABB**

3.2 Question 2 — Download URL

Windows tags every file downloaded from the internet with a hidden NTFS Alternate Data Stream named Zone.Identifier, which records the originating URL and referrer. This ADS was located and inspected directly in FTK Imager against the downloaded executable.

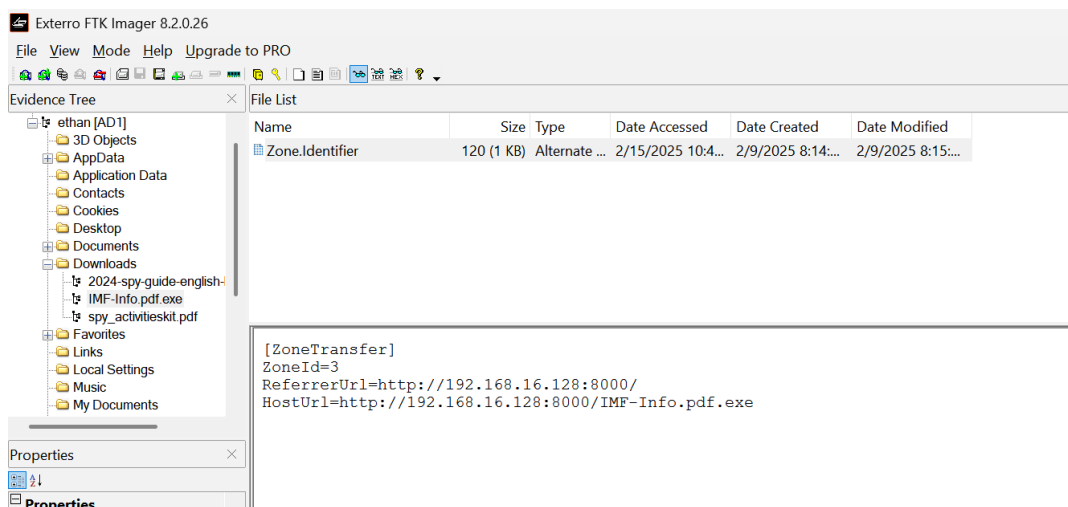


Figure 3 — Zone.Identifier ADS on IMF-Info.pdf.exe, showing ReferrerUrl and HostUrl pointing to the attacker-controlled server.

Question 2

What is the URL from which the file was downloaded?

Answer: <http://192.168.16.128:8000/IMF-Info.pdf.exe>

3.3 Question 3 — Delivery Application

Two browser profiles were present on the host: a Chrome profile (History) and an Edge profile (History.copy0). Each profile's SQLite database was opened in DB Browser for SQLite to inspect the downloads and downloads_url_chains tables, which record every file download and its associated URL redirect chain.

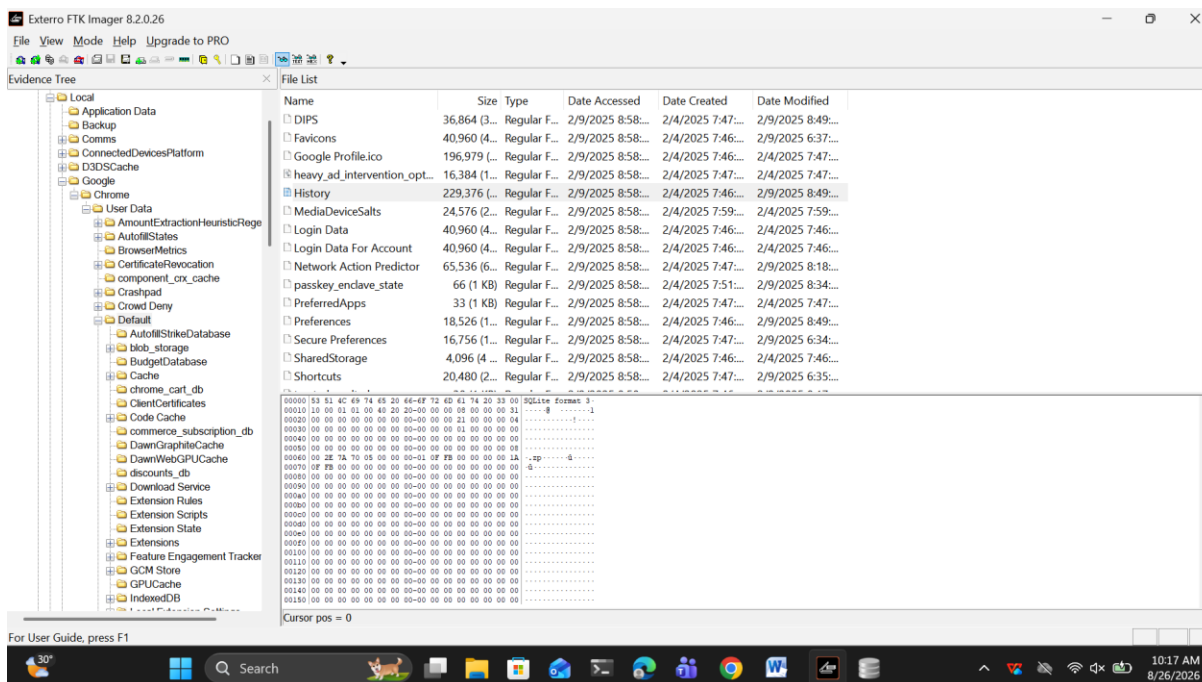


Figure 4 — FTK Imager: the Chrome User Data profile structure recovered from the image.

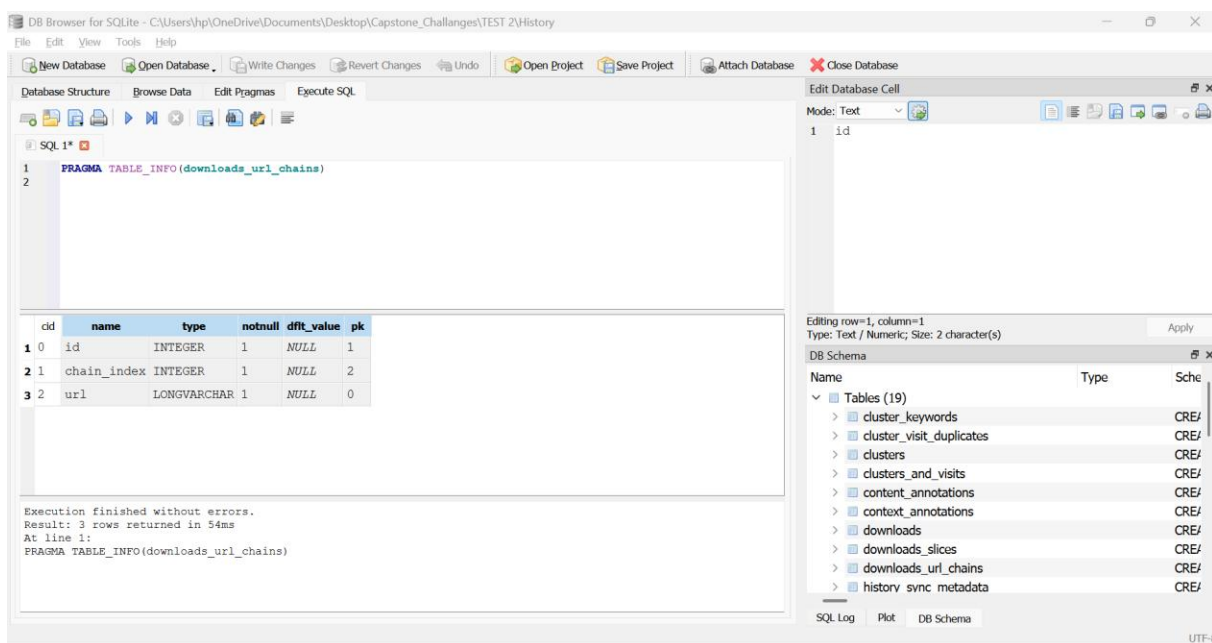


Figure 5 — DB Browser for SQLite: schema of the downloads_url_chains table (id, chain_index, url).

Querying the second profile's history database (History.copy0), which corresponds to the Microsoft Edge browser (confirmed by the edge_favicons table visible in that database's schema), showed the complete download chain for the malicious executable — along with two related PDF downloads (IMF-Secret.pdf and IMF-Mission.pdf) from the same attacker-controlled host, both of which were later encrypted by the malware.

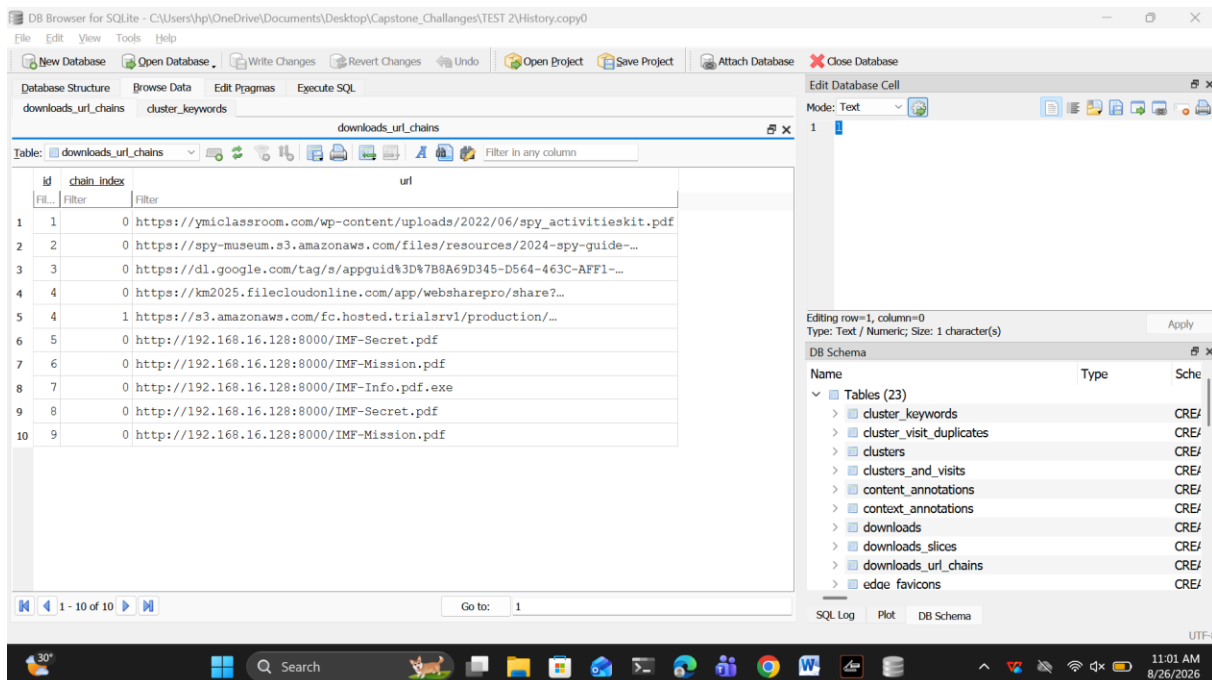


Figure 6 — Edge history (History.copy0), downloads_url_chains table: IMF-Info.pdf.exe, IMF-Secret.pdf, and IMF-Mission.pdf all downloaded from 192.168.16.128:8000.

Question 3

What application did the user use to download this file?

Answer: Microsoft Edge

3.4 Question 4 — Compromised / At-Risk Server IPs (Windows Mail)

The modern Windows Mail app stores cached message data in an ESE (Extensible Storage Engine) database named HxStore.hxd, located under the user's

AppData\Local\Packages\microsoft.windowscommunicationsapps_...\\LocalState\\ folder. Rather than fully parsing the ESE structure, a direct regular-expression sweep for IPv4 addresses was run against the raw file, since message body fragments are frequently recoverable as plaintext strings even without a structured parse.

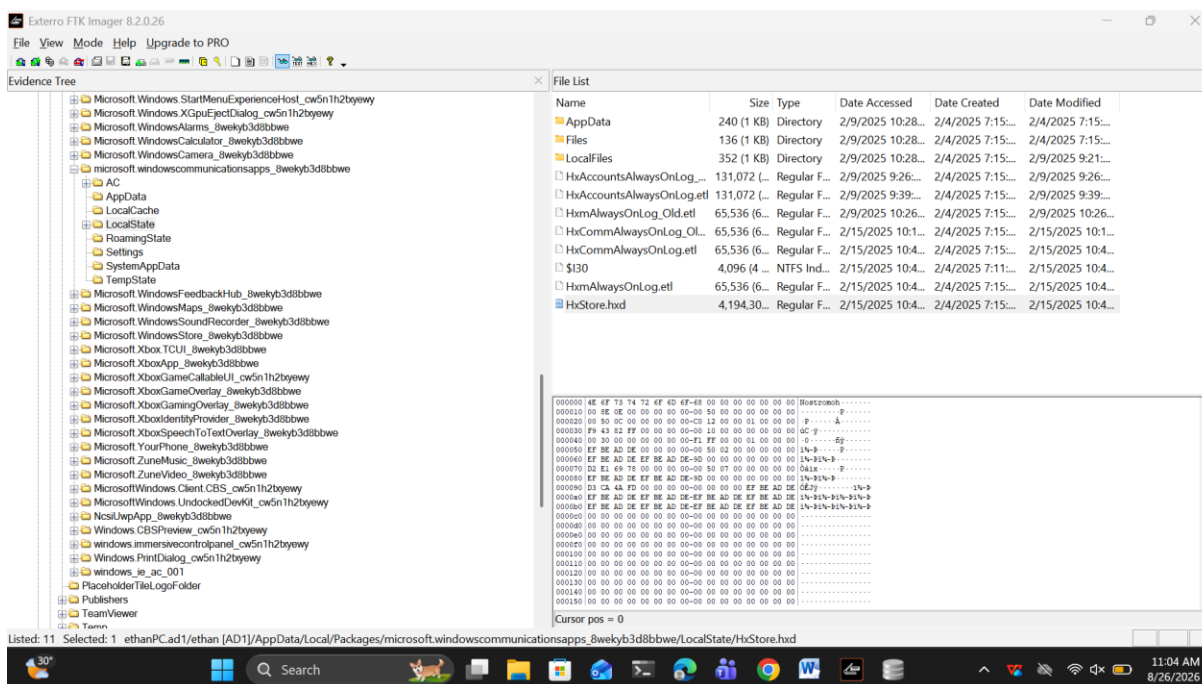


Figure 7 — FTK Imager: location of HxStore.hxd inside the Windows Mail app's LocalState folder.

```
Select-String -Path "hxstore.hxd" -Pattern "(\\d{1,3}\\.){3}\\d{1,3}" -Encoding UTF8 -AllMatches |
ForEach-Object { $_.Matches.Value } | Sort-Object -Unique
```

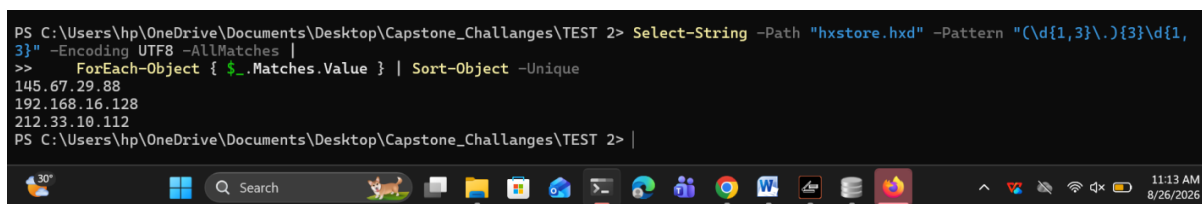


Figure 8 — Three unique IP addresses recovered from an email body embedded in HxStore.hxd.

Question 4

By examining Windows Mail artifacts, an email was found mentioning three IP addresses of servers that are at risk or compromised. What are the IP addresses?

Answer: 145.67.29.88 | 192.168.16.128 | 212.33.10.112

3.5 Question 5 — Predefined Encryption Password

Inspection of IMF-Info.pdf.exe's internal build paths (e.g. AppData\Local\Temp\pkg.<hash>\node\src...) confirmed it is a Node.js application bundled into a standalone Windows executable using the pkg packager. Because pkg embeds the application's JavaScript source largely as readable text inside the compiled binary, targeted string searches were used instead of dynamic execution or a debugger.

```
Select-String -Path "IMF-Info.pdf.exe" -Pattern "powershell|FromBase64|IEX|-enc|ConvertTo-  
SecureString" -Encoding UTF8 -AllMatches
```

This surfaced the exact trigger logic: the program writes a temporary .ps1 file to disk and launches it with:

```
powershell.exe -ExecutionPolicy Bypass -File "<tempfile>.ps1"
```

A follow-up search located the actual payload assembled just before that call — a large string that is reversed character-by-character, Base64-decoded, and then run via a renamed alias of Invoke-Expression (PwN):

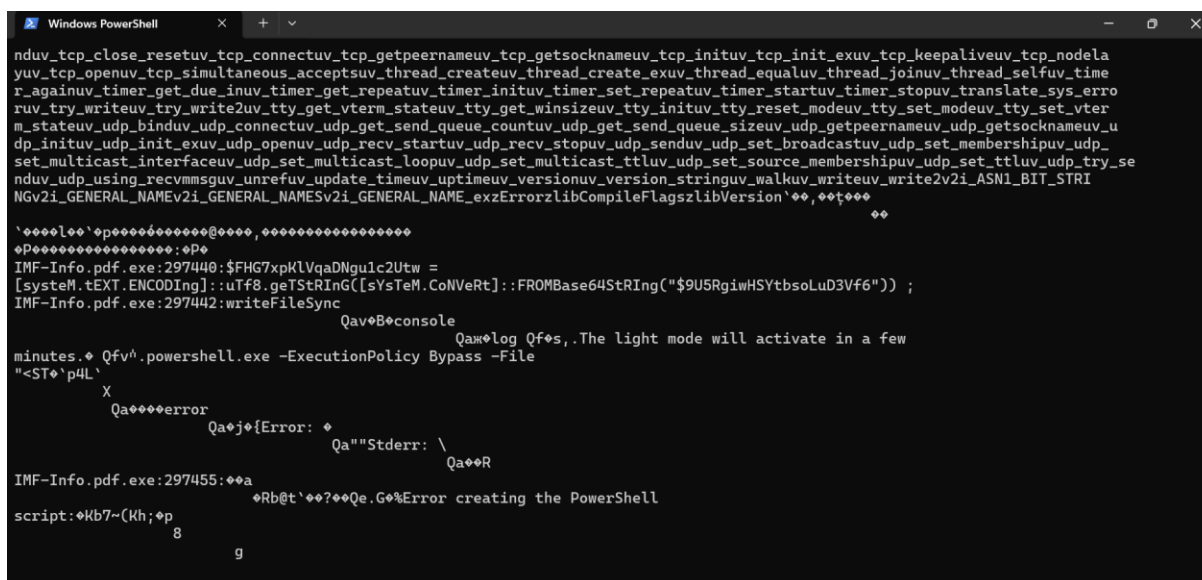


Figure 9 — Extracted strings showing the writeFileSync / powershell.exe -ExecutionPolicy Bypass -File invocation.

[illegible]

Figure 10 — The obfuscated payload variable, later reversed and Base64-decoded, immediately preceding the `writeFileSync` call.

The obfuscated blob was deobfuscated using CyberChef with a two-step recipe — Reverse (Character) followed by From Base64 — exactly mirroring the malware's own logic, without ever executing the payload:

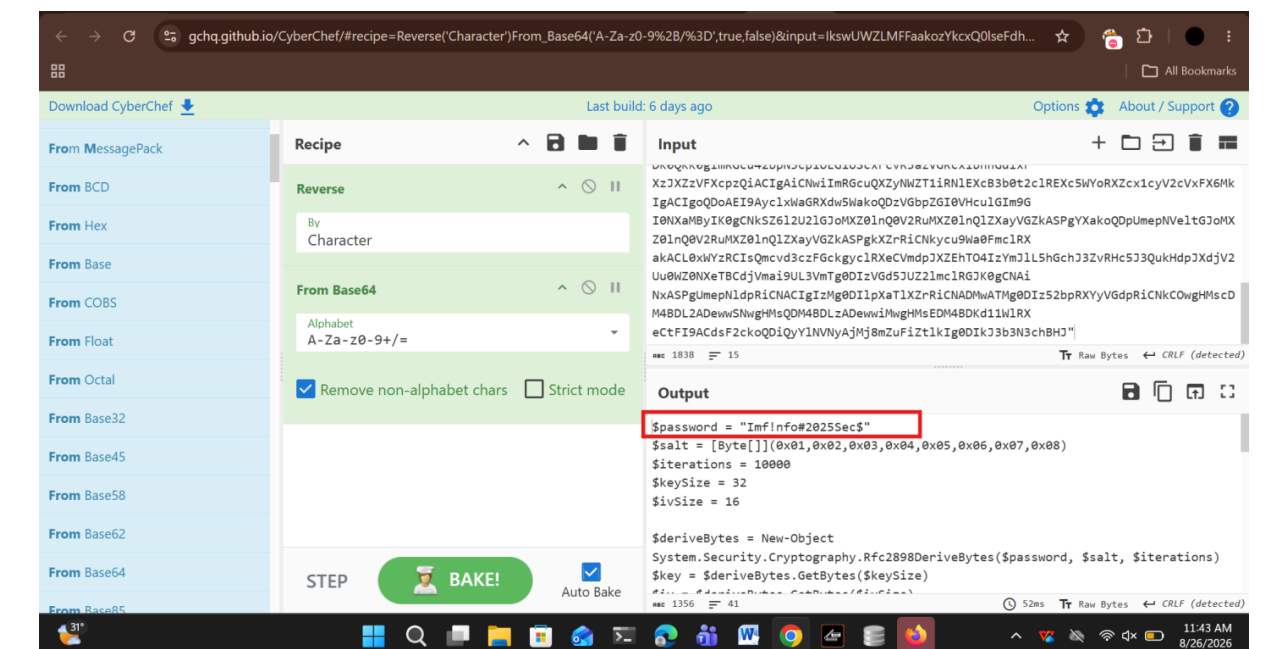


Figure 11 — CyberChef recipe (Reverse → From Base64) revealing the plaintext PowerShell encryption script, including the hardcoded password.

The recovered script performs the following:

- Derives a 256-bit AES key and 128-bit IV from a hardcoded password using PBKDF2 (Rfc2898DeriveBytes), a fixed 8-byte salt, and 10,000 iterations.

- Reads C:\Users\ethan\Desktop\IMF-Secret.pdf and C:\Users\ethan\Desktop\IMF-Mission.pdf.
- Encrypts each file with AES-256-CBC (PKCS7 padding) to a matching .enc file.
- Deletes the original plaintext PDFs, leaving only the encrypted copies behind.

Question 5

By examining the malicious executable, it was found to use an obfuscated PowerShell script to decrypt specific files. What predefined password does the script use for encryption?

Answer: Imf!nfo#2025Sec\$

3.6 Question 6 — Decryption and Recovered Secret

With the algorithm (AES-256-CBC, PKCS7 padding), key-derivation scheme (PBKDF2, salt 01 02 03 04 05 06 07 08, 10,000 iterations, 32-byte key / 16-byte IV) and password all recovered from the malware's own script, the two .enc files located on the subject's Desktop within the disk image were exported and decrypted using a native PowerShell re-implementation of the same routine, run in reverse (CreateDecryptor instead of CreateEncryptor).

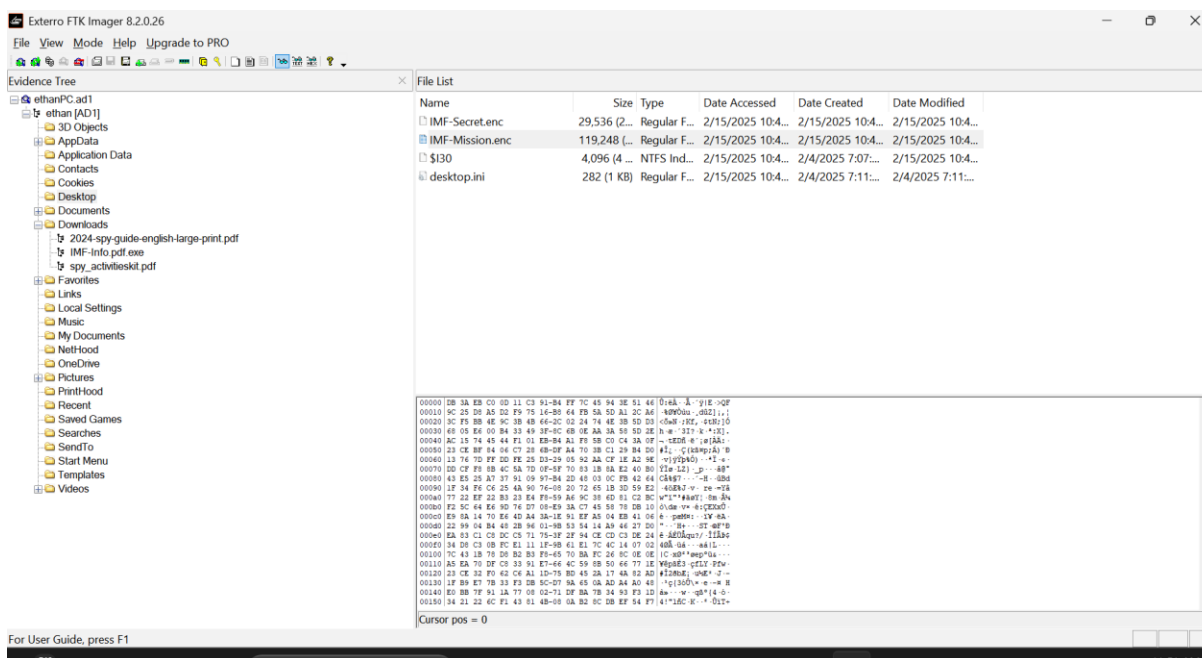


Figure 12 — FTK Imager: IMF-Secret.enc and IMF-Mission.enc located on ethan's Desktop within the mounted evidence.

```

Windows PowerShell
Directory: C:\Users\hp\OneDrive\Documents\Desktop\Capstone_Challenges\TEST 2

Mode                LastWriteTime         Length Name
----                -
d-----          8/25/2026   9:46 AM                PASS@cyberdefenders.org
d-----          8/26/2026   1:05 AM                Silent_Breach
-a-----          8/26/2026  10:38 AM             229376 History
-a-----          2/16/2025   4:10 AM             458752 History.copy0
-a-----          2/16/2025   4:16 AM             4194304 HxStore.hxd
-a-----          2/10/2025   1:45 AM             34858017 IMF-Info.pdf.exe
-a-----          2/16/2025   4:15 AM             119248 IMF-Mission.enc
-a-----          2/16/2025   4:15 AM             29536 IMF-Secret.enc
-a-----          8/25/2026   9:41 AM             671782500 Silent_Breach.zip
-a-----          8/25/2026   9:42 AM              9716 Silent_Breach_Challenge.docx

PS C:\Users\hp\OneDrive\Documents\Desktop\Capstone_Challenges\TEST 2> Get-ChildItem -Recurse -Include "*.enc","IMF-Secret*","IMF-Mission*" -ErrorAction SilentlyContinue

Directory: C:\Users\hp\OneDrive\Documents\Desktop\Capstone_Challenges\TEST 2

Mode                LastWriteTime         Length Name
----                -
-a-----          2/16/2025   4:15 AM             119248 IMF-Mission.enc
-a-----          2/16/2025   4:15 AM             29536 IMF-Secret.enc

PS C:\Users\hp\OneDrive\Documents\Desktop\Capstone_Challenges\TEST 2> function Decrypt-File {
>> param(

```

Figure 13 — Both .enc files located and exported to the working case folder for decryption.

```

function Decrypt-File {    param([string]$InputFile, [string]$OutputFile)    $password =
"Imf!Info#2025Sec$"    $salt = [Byte[]](0x01,0x02,0x03,0x04,0x05,0x06,0x07,0x08)    $iterations
= 10000    $deriveBytes = New-Object System.Security.Cryptography.Rfc2898DeriveBytes($password,
$salt, $iterations)    $key = $deriveBytes.GetBytes(32)    $iv = $deriveBytes.GetBytes(16)
$aes = [System.Security.Cryptography.Aes]::Create()    $aes.Key = $key; $aes.IV = $iv
$aes.Mode = [System.Security.Cryptography.CipherMode]::CBC    $aes.Padding =
[System.Security.Cryptography.PaddingMode]::PKCS7    $decryptor = $aes.CreateDecryptor()
$cipherBytes = [System.IO.File]::ReadAllBytes($InputFile)    $memStream = New-Object
System.IO.MemoryStream    $cryptoStream = New-Object
System.Security.Cryptography.CryptoStream($memStream, $decryptor,
[System.Security.Cryptography.CryptoStreamMode]::Write)    $cryptoStream.Write($cipherBytes, 0,
$cipherBytes.Length)    $cryptoStream.FlushFinalBlock()
[System.IO.File]::WriteAllBytes($OutputFile, $memStream.ToArray()) }

```

```

PS C:\Users\hnp\OneDrive\Documents\Desktop\Capstone_Challenges\TEST 2> function Decrypt-File {
>> param(
>> [string]$InputFile,
>> [string]$OutputFile
>> )
>>
>> $password = "Imf!nfo#2025Sec$"
>> $salt = [Byte[]](0x01,0x02,0x03,0x04,0x05,0x06,0x07,0x08)
>> $iterations = 10000
>> $keySize = 32
>> $ivSize = 16
>>
>> $deriveBytes = New-Object System.Security.Cryptography.Rfc2898DeriveBytes($password, $salt, $iterations)
>> $key = $deriveBytes.GetBytes($keySize)
>> $iv = $deriveBytes.GetBytes($ivSize)
>>
>> $aes = [System.Security.Cryptography.Aes]::Create()
>> $aes.Key = $key
>> $aes.IV = $iv
>> $aes.Mode = [System.Security.Cryptography.CipherMode]::CBC
>> $aes.Padding = [System.Security.Cryptography.PaddingMode]::PKCS7
>>
>> $decryptor = $aes.CreateDecryptor()
>> $cipherBytes = [System.IO.File]::ReadAllBytes($InputFile)
>>
>> $memStream = New-Object System.IO.MemoryStream
>> $cryptoStream = New-Object System.Security.Cryptography.CryptoStream($memStream, $decryptor, [System.Security.Cryptography.CryptoStreamMode]::Write)
>> $cryptoStream.Write($cipherBytes, 0, $cipherBytes.Length)
>> $cryptoStream.FlushFinalBlock()
>>
>> [System.IO.File]::WriteAllBytes($OutputFile, $memStream.ToArray())

```

Figure 14 — The AES-256-CBC decryption routine executed in PowerShell against both .enc files.

Both files decrypted cleanly to valid PDFs. IMF-Mission.pdf (“Operation Shadow Veil” mission briefing) contained the secret flag string in its closing section:

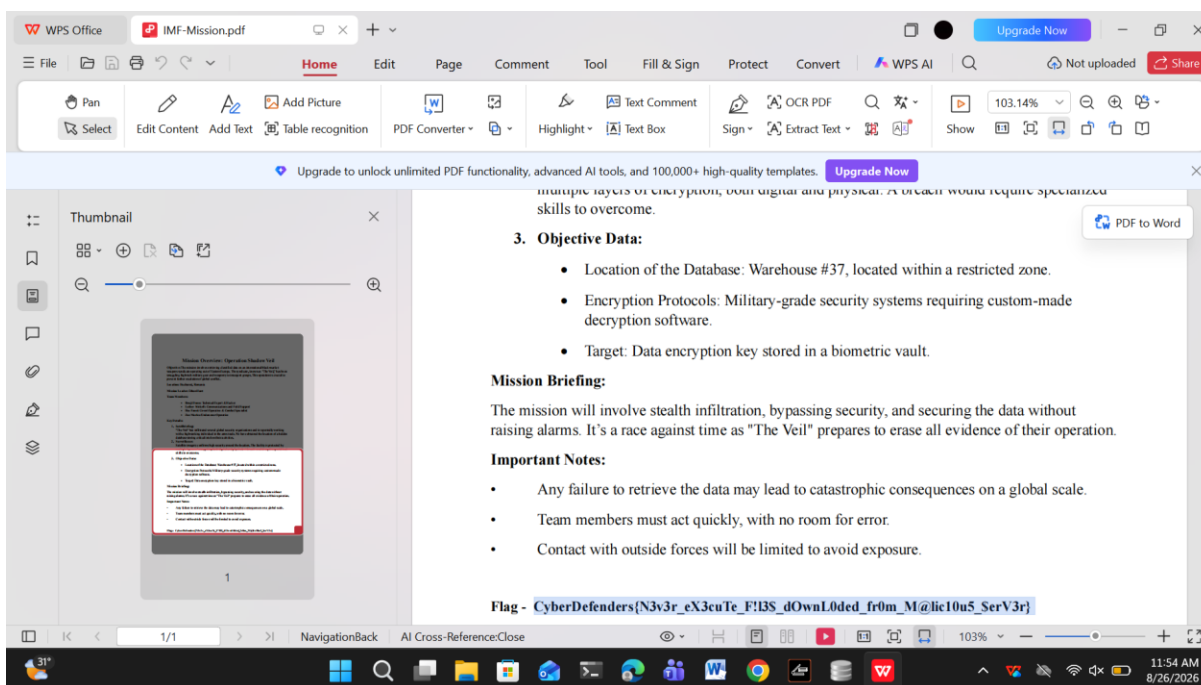


Figure 15 — Decrypted IMF-Mission.pdf opened, showing the recovered flag.

Question 6

After identifying how the script works, decrypt the files and submit the secret string (flag format).

Answer: `CyberDefenders{N3v3r_eX3cuTe_F!l3$_d0wnL0ded_fr0m_M@lic10u5_$erv3r}`

4. Indicators of Compromise (IOCs)

The following indicators were collected throughout the investigation and should be used for detection, blocking, and threat-intel correlation.

Indicator	Type	Context
336A7CF476EBC7548C93507339196ABB	MD5 Hash	Hash of the malicious dropper, IMF-Info.pdf.exe
IMF-Info.pdf.exe	Filename	Double-extension disguise (.pdf.exe) used to masquerade as a PDF document
http://192.168.16.128:8000/IMF-Info.pdf.exe	URL	Direct download URL for the malicious executable
192.168.16.128	IP Address	Hosting server for the malware and staged PDFs; also referenced as an at-risk server in Windows Mail
145.67.29.88	IP Address	At-risk / compromised server referenced in a Windows Mail message
212.33.10.112	IP Address	At-risk / compromised server referenced in a Windows Mail message
Imf!nfo#2025Sec\$	Credential / Key material	Hardcoded password used for PBKDF2 key derivation in the malware's AES encryption routine
powershell.exe -ExecutionPolicy Bypass -File	Command line	Execution pattern used by the dropper to run its self-written PowerShell payload

5. MITRE ATT&CK Mapping

Techniques observed during this investigation, mapped to the MITRE ATT&CK Enterprise framework:

Tactic	Technique	ID	Evidence
Initial Access	Drive-by Compromise / User Download	T1189	Executable downloaded directly by the user via Edge browser from an attacker-hosted server
Defense Evasion	Masquerading (double file extension)	T1036.008	File named IMF-Info.pdf.exe to visually disguise a PE executable as a PDF document
Execution	Command and Scripting Interpreter: PowerShell	T1059.001	Dropper writes and launches a PowerShell script via powershell.exe -ExecutionPolicy Bypass -File
Defense Evasion	Deobfuscate/Decode Files or Information	T1140	Payload stored reversed and Base64-encoded inside the binary; decoded at runtime before execution
Impact	Data Encrypted for Impact	T1486	AES-256-CBC encryption of IMF-Secret.pdf and IMF-Mission.pdf using a PBKDF2-derived key, followed by deletion of the originals
Collection	Data from Local System	T1005	Malware targets specific user files (Desktop PDFs) for encryption/exfil-style impact

6. Conclusion & Recommendations

This investigation confirmed a full, self-contained infection chain: a user downloaded a disguised executable via Microsoft Edge from an internal-looking staging server, executed it, and the program silently deployed a PowerShell-based file-encryption routine against sensitive local documents. All six investigative questions were answered using non-destructive, static forensic techniques — disk image review, browser and mail artifact analysis, and string-level malware analysis — without ever executing the malicious binary.

Recommendations

- Block outbound connections to 192.168.16.128, 145.67.29.88, and 212.33.10.112 and add the recovered MD5 hash to endpoint detection blocklists.
- Enforce policies that prevent execution of files with disguised/double extensions (e.g. *.pdf.exe) and enable Windows “Show file extensions” by default.
- Apply PowerShell Constrained Language Mode and script-block logging to detect Invoke-Expression-based obfuscated execution patterns.
- Educate users on verifying download sources before executing files, particularly ones received or staged via non-standard internal IPs and ports.
- Maintain offline/immutable backups of sensitive documents to reduce impact from future file-encryption incidents.